

Makemore

Part 2 — The MLP (Bengio et al. 2003)

A complete, cell-by-cell study handbook for the second video in Andrej Karpathy's *makemore* series — “Building makemore Part 2: MLP.”

Compiled by: **Shreyas Pradeepkumar Khandale** · GitHub: *sherurox*

Companion to: `makemore2.ipynb`

Where we are in the series

Part 1 built a bigram model two ways — by counting and by training a tiny neural network — and showed they converge to the same matrix. The final loss was about **2.45**, and the names sampled from it (*mor*, *axx*, *minaymoryles*) sounded vaguely like names but were obviously broken. The problem was structural: with only one character of context, the model cannot see far enough to write good names.

Part 2 fixes that. We follow Bengio et al.'s 2003 paper *A Neural Probabilistic Language Model* — the paper that introduced learned word embeddings — and build a multi-layer perceptron that takes **three previous characters** as context. We end up with a loss of **2.20**, a real improvement, and we learn most of the deep-learning workflow vocabulary along the way: embeddings, hidden layers, `F.cross_entropy`, minibatches, learning rate finders, learning rate decay, and the train/dev/test split.

What is actually new

- **Embeddings.** Instead of one-hot vectors, each character now lives in a learned low-dimensional space (a vector of 2, then 10 floats). This is the same primitive that scales all the way up to GPT.
- **Context windows.** The input is a sliding window of `block_size` previous characters, not just one. This is the model's “memory.”
- **A real hidden layer.** We add a `tanh` non-linearity between two linear layers. Now the network can model interactions between context positions.
- `F.cross_entropy`. Replaces our hand-rolled `softmax + NLL` with a single fused, numerically-stable call.
- **Minibatches.** We stop computing the full-batch gradient on 228k examples per step and use 32-example minibatches. Less accurate per step, but vastly more steps per second.

- **Learning rate search.** A clever trick using `torch.linspace` to discover a good learning rate empirically rather than guessing.
- **Train/dev/test split.** Without this, you cannot tell underfitting from overfitting, and tuning hyperparameters becomes superstition.

Contents

1. Why we have to move beyond the bigram
2. The Bengio 2003 paper at a glance
3. Setup: imports, data, stoi and itos
4. Building the dataset with context windows
5. The embedding matrix C
6. Embedding lookup: indexing equals one-hot matmul
7. Embedding the entire dataset in one operation
8. The hidden layer: W1, b1
9. The flattening problem — (32, 3, 2) is not (32, 6)
10. Three ways to flatten: cat, unbind+cat, .view
11. Why .view is the winner
12. The tanh non-linearity
13. The output layer and logits
14. Manual softmax and NLL — the slow way
15. Making it respectable — parameter lists and generators
16. F.cross_entropy — efficiency and numerical stability
17. The first training loop — overfitting on purpose
18. Scaling up: full dataset of 228k bigrams
19. Minibatches with torch.randint
20. The learning rate finder
21. Reading the LR-vs-loss plot — choosing 0.1
22. Learning rate decay
23. The train / dev / test split — what each one is for
24. Underfitting: when train loss equals dev loss
25. First scale-up: 100 → 300 hidden neurons
26. Visualizing the 2-D embedding space
27. Second scale-up: 2-D → 10-D embeddings
28. Beating the bigram baseline — and the first sign of overfitting
29. Summary and key takeaways
30. Glossary

1. Why we have to move beyond the bigram

Karpathy opens the video with a direct admission: the bigram model is bad. Not because the implementation was wrong, but because the architecture itself cannot see far enough to write believable names. *mor*, *axx*, and *minaymoryles* are the best it could do.

The natural fix — use more context — runs into a wall when you try to do it by counting:

Context size	Combinations to count	Status
1 char (bigram)	27	fine — the Part 1 model
2 chars (trigram)	$27^2 = 729$	still fine
3 chars	$27^3 = 19,683$	barely tractable
5 chars	$27^5 \approx 14$ million	most combinations never seen
10 chars	$27^{10} \approx 2 \times 10^{16}$	impossible

The counting table grows *exponentially* with context length. Most cells would be zero (you never saw *jqxqz* in your data) so probabilities would be undefined or have to be smoothed into uselessness. **The counting approach fundamentally cannot scale.**

The pivot

We need a model that can take a longer context but whose number of parameters does *not* blow up exponentially with context length. The Bengio 2003 paper is the canonical answer: represent each token as a learned dense vector, concatenate those vectors, and feed them through a neural net. Parameter count then grows *linearly* with context length, not exponentially.

2. The Bengio 2003 paper at a glance

Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Jauvin published *A Neural Probabilistic Language Model* in JMLR 2003. It is the paper that introduced learned embeddings to language modelling, and the architectural blueprint we follow in this video is almost exactly theirs — just shrunk from words to characters.

Their architecture

- **Vocabulary:** 17,000 English words (we use 27 characters).
- **Embedding dimension:** 30 floats per word (we will try 2, then 10).
- **Context size:** 3 previous words (we will use 3 previous characters).
- **Hidden layer:** one fully-connected layer with tanh non-linearity, size is a hyperparameter (they tried a few; we will try 100, then 300, then 200).

- **Output layer:** 17,000-way softmax (we will have 27-way).
- **Loss:** “maximize the log-likelihood of the training data” — same objective as Part 1.

Why embeddings are the big idea

Bengio's key claim was that words with similar meanings should end up with similar embedding vectors — *sofa* and *chair* close together in the 30-D space, far from *airplane*. The embeddings are randomly initialized and then learned by backprop, alongside everything else. This means the model can generalize: even if it never saw “*the cat sat on the sofa*,” if it saw “*the cat sat on the chair*” and learned that sofa and chair are close, it can transfer.

For characters this matters less semantically (there is no meaningful difference between letters), but the machinery is identical. And it is the machinery that scales all the way to GPT's token embeddings.

3. Setup: imports, data, stoi and itos

The first few cells are essentially copied from Part 1. The vocabulary is unchanged (27 tokens), and `names.txt` is the same file.

Cell — Imports

```
import torch
from torch.nn import functional as F
import matplotlib.pyplot as plt
%matplotlib inline
```

`torch.nn.functional` is imported as `F` — this is the standard convention. Most of the building blocks we will use (`F.one_hot`, `F.cross_entropy`, `F.tanh`) live there.

Cells — Load words and build the character lookup

```
words = open('names.txt', 'r').read().splitlines()
print(words[:8])
len(words)           # 32033
```

```
chars = sorted(list(set(''.join(words))))
stoi = {s: i+1 for i, s in enumerate(chars)}
stoi['.'] = 0
itos = {i: s for s, i in stoi.items()}
```

Identical to Part 1. `stoi` maps each lowercase letter to an integer 1–26, with the special boundary token `.` at index 0. `itos` is the reverse.

4. Building the dataset with context windows

This is the first real architectural change. We are going to predict the next character from the previous `block_size` characters, so we need a dataset of (*context*, *target*) pairs.

Cell — Build (X, Y) with block_size = 3

```
block_size = 3           # context length: how many previous chars
X, Y = [], []
```

```

for w in words:
    context = [0] * block_size      # start with [., ., .]
    for ch in w + '.':
        ix = stoi[ch]
        X.append(context)
        Y.append(ix)
        context = context[1:] + [ix] # slide the window

X = torch.tensor(X)
Y = torch.tensor(Y)

```

`block_size = 3` is the only hyperparameter that controls how much context the model sees. Three is what Bengio used; we will keep it here.

Tracing through *emma* with `block_size 3`:

Context	Target	Meaning
[. , . , .]	. e	name starts — predict 'e'
[. , . , e]	e m	after 'e', predict 'm'
[. , e , m]	m m	after 'em', predict 'm'
[e , m , m]	m a	after 'emm', predict 'a'
[m , m , a]	a .	after 'mma', predict end

- `context = [0] * block_size` — initialize with `[0, 0, 0]`, three copies of the boundary token `..`. This is how the model knows we are at the start of a word.
- `for ch in w + '.'` — iterate through each character of the word and the trailing end-token. Each name produces $\text{len}(\text{name}) + 1$ training examples.
- `context = context[1:] + [ix]` — slide the window: drop the leftmost element, append the newly-emitted character on the right. Crop and append.

Cell — Sanity-check shapes

```

X.shape, X.dtype, Y.shape, Y.dtype
# (torch.Size([32, 3]), torch.int64, torch.Size([32]), torch.int64)
# (using only words[:5] for the first experiments)

```

Initially Karpathy works with only the first 5 words to keep numbers small and printable. The first 5 names produce 32 (context, target) pairs — that is where the 32 comes from. Later, when we use all 32,000 words, the dataset grows to 228,146 examples.

5. The embedding matrix C

Each of the 27 characters needs to be turned into a vector before it can flow through the network. In Part 1 we used a one-hot vector (a 27-D vector with a single 1). That works but wastes 26 dimensions per character. Bengio's idea: learn a *dense* low-dimensional vector for each token. Two dimensions to start — small enough to plot.

Cell — Initialize the embedding matrix

```
C = torch.randn((27, 2))
```

C is a 27×2 tensor. Row i is the 2-D embedding vector for character i . At initialization the rows are just random normal samples; the network will tune them during training to whatever makes the loss low.

Why 2 dimensions?

Pedagogy. Karpathy picks 2 so we can plot the embeddings on a flat scatter plot at the end of the lecture and literally see whether vowels cluster together. For real models 2 is far too few — by the end of this notebook we will move to 10 dimensions, and modern LLMs use thousands.

6. Embedding lookup: indexing equals one-hot matmul

We need to convert an integer character index into its 2-D embedding vector. PyTorch makes this trivially easy. Three equivalent ways, demonstrated cell-by-cell:

Cell — Plain indexing

```
C[5]
# returns the 5th row of C — the 2-D embedding for character 'e'
```

Cell — Indexing with a tensor of indices

```
C[torch.tensor([5, 6, 7])]
# returns a (3, 2) tensor — three rows stacked
```

Cell — The one-hot route, for proof of equivalence

```
F.one_hot(torch.tensor(5), num_classes=27).float() @ C
# returns the same 2-D vector as C[5]
```

All three lines return the same vector. The third one is the formal “textbook” way: turn the index 5 into a 27-D one-hot vector, multiply by C. Because the one-hot vector has a 1 at position 5 and zeros elsewhere, the matmul plucks out row 5 of C. It is the same operation. The indexing form is just dramatically faster — no one-hot tensor is ever materialized.

The hidden continuity from Part 1

In Part 1 we built a one-hot vector and matmul'd by W to get logits. That matmul was also “just a row lookup.” The MLP's embedding layer is exactly the same operation, just stripped of the fiction of a one-hot vector. Embeddings are not a new concept — they are the natural way to write what we were already doing.

7. Embedding the entire dataset in one operation

The same indexing magic works with multi-dimensional integer tensors. We can embed every character in every context of the entire dataset in a single line:

Cell — Embed all of X

```
C[X].shape
# torch.Size([32, 3, 2])
```

X has shape (32, 3) — 32 examples, 3 characters of context each. C[X] has shape (32, 3, 2) — for every example and every context position, we now have a 2-D embedding vector. PyTorch broadcasts the indexing operation across all dimensions of X.

Cell — Verify the lookup did what we think

```
X[13, 2]          # the integer at row 13, column 2 of X
C[X][13, 2]       # the embedding vector at that position
C[1]              # the embedding for character index 1
                  # if X[13, 2] == 1, then C[X][13, 2] == C[1]
```

Cell — Stash it

```
emb = C[X]
emb.shape          # (32, 3, 2)
```

emb is now a 3-D tensor we will feed into the rest of the network. The comment in your notebook captures it well: *“PyTorch embedding is awesome to embed everything simultaneously.”*

8. The hidden layer: W1, b1

After the embedding layer, Bengio's architecture has a single fully-connected hidden layer with a tanh non-linearity. We set up its parameters:

Cell — Initialize hidden-layer parameters

```
W1 = torch.randn((6, 100))  # 3 * 2 = 6 inputs, 100 hidden neurons
b1 = torch.randn(100)       # one bias per neuron
```

Why (6, 100)?

- **6 input dimensions:** we have `block_size = 3` context positions, each turned into a 2-D embedding. So the concatenated input to this layer is $3 \times 2 = 6$ numbers.
- **100 output dimensions:** this is the hidden-layer width, a hyperparameter chosen by us. Bigger = more capacity, more parameters, slower training, easier to overfit. Karpathy starts at 100, later bumps it to 300, then settles on 200 once embeddings are also bigger.
- **The bias vector b1 has shape (100,)** — one bias for each hidden neuron, broadcast across the batch dimension during the forward pass.

9. The flattening problem — (32, 3, 2) is not (32, 6)

Here is the snag. emb has shape (32, 3, 2), but to multiply it by W1 of shape (6, 100) we need the input to be shape (32, 6). Matmul rules do not let us multiply a 3-D tensor by a 2-D tensor in the way we want.

Cell — Try the naive thing (this almost works, but not quite)

```
emb @ W1 + b1
# RuntimeError or unexpected broadcasting – the shapes do not line up
```


We need to *flatten* the middle two dimensions: combine the “3 context positions” and “2 embedding dims per position” into a single “6 numbers per example” dimension. PyTorch gives us several ways to do this, and the lecture walks through each one to teach the API.

10. Three ways to flatten: cat,.unbind+cat, .view

Way 1 — torch.cat with hard-coded indices

```
torch.cat([emb[:, 0, :], emb[:, 1, :], emb[:, 2, :]], 1).shape
# torch.Size([32, 6])
```

We manually grab each context position — `emb[:, 0, :]` is the embedding for the first context position across all 32 examples, shape (32, 2) — and concatenate them along dimension 1. Works, but ugly: change `block_size` from 3 to 5 and you have to rewrite this line.

Way 2 — torch.unbind

```
torch.cat(torch.unbind(emb, 1), 1).shape
# torch.Size([32, 6])
```

`torch.unbind(emb, 1)` removes dimension 1 and returns a tuple of slices: (`emb[:,0,:]`, `emb[:,1,:]`, `emb[:,2,:]`). We cat that tuple along dim 1. This generalizes — does not matter if `block_size` is 3 or 5 or 50. But it still allocates new memory and copies data around.

Way 3 — .view (the winner)

```
emb.view(32, 6)          # explicit
emb.view(-1, 6)          # let PyTorch infer the batch dimension
```

`.view` reinterprets the same underlying memory as a different shape, with no copy. Your notebook verifies the equivalence:

```
emb.view(32, 6) == torch.cat(torch.unbind(emb, 1), 1)
# tensor of all True
```

11. Why .view is the winner

PyTorch tensors have two layers: the data itself (a flat 1-D block of memory) and the metadata describing how to interpret that block as a multi-dimensional array. Shape, stride, offset — that metadata. `.view` changes only the metadata. No data is moved, no memory is allocated. It is effectively free.

Reading tip from Karpathy

He says: “You should read this excellent blog post by Edward Yang from PyTorch — ‘PyTorch internals.’ It explains exactly how the storage and the view-metadata interact.” This is the reason `.view` is so much faster than `cat` or `unbind`: it is not a real operation, it is a relabelling.

Two more idioms worth memorizing:

- **The -1 trick.** `emb.view(-1, 6)` tells PyTorch “figure out this dimension from the total number of elements.” You only get to use -1 once per call. Use it for the batch dimension so the code works for any batch size.
- **.view vs .reshape.** They look similar; `.view` requires the tensor to be contiguous in memory and is always a no-copy, while `.reshape` falls back to a copy if needed. Karpathy uses `.view` because he wants the no-copy guarantee, and the tensors here always happen to be contiguous.

Cell — Wire up the hidden activations

```
h = torch.tanh(emb.view(-1, 6) @ W1 + b1)
h.shape    # (32, 100) — 100-D activations for each of 32 examples
```

12. The tanh non-linearity

`torch.tanh` applies the hyperbolic tangent element-wise. It squashes any real number into the range $(-1, 1)$: big positive numbers saturate near +1, big negative numbers saturate near -1, and small values pass through nearly linearly.

Why we need a non-linearity at all

If we stacked two linear layers without anything between them, the composition $(x @ W1 + b1) @ W2 + b2$ is mathematically equivalent to a single linear layer with weight $W1 @ W2$. No matter how deep we stack, the network can only represent linear functions of the input. **The non-linearity is what makes “deep” deep.**

Why tanh specifically

`tanh` was the default in 2003 because it is smooth (differentiable everywhere) and zero-centered. Modern nets typically use ReLU and its variants, which are cheaper to compute and avoid the “vanishing gradient” problem that haunts deep tanh networks (Karpathy spends time on this in Part 3, where we deliberately stress-test the initialization). For Bengio’s shallow MLP, tanh is fine.

13. The output layer and logits

Cell — Initialize the output layer

```
W2 = torch.randn((100, 27))
b2 = torch.randn(27)
```

The output is a 27-way classification — one score per possible next character. So `W2` is shape $(100, 27)$ (100 hidden units in, 27 outputs) and `b2` is shape $(27,)$.

Cell — Compute the logits

```
logits = h @ W2 + b2
logits.shape    # (32, 27)
```

“Logits” is the standard name for the raw, pre-softmax output of a classifier. Same interpretation as Part 1: these are *log-counts*. The next two lines turn them into probabilities.

14. Manual softmax and NLL — the slow way

Just to show the equivalence to Part 1, the notebook first does the softmax + NLL by hand:

Cell — Manual softmax

```
counts = logits.exp()                # exp turns log-counts into counts
prob = counts / counts.sum(1, keepdims=True)  # row-normalize → probabilities
prob.shape                          # (32, 27)
```

Cell — Pluck out the probability assigned to the correct next char

```
Y                                # the 32 correct next-character indices
torch.arange(32)                 # 0, 1, 2, ..., 31
loss = -prob[torch.arange(32), Y].log().mean()
loss                            # tensor(17.7697)
```

Exactly the same vectorized NLL line from Part 1. For each of the 32 examples, look up `prob[i, Y[i]]` — the probability the network assigned to the correct answer — take its log, average, negate.

Why is the initial loss 17.77 and not $\log(27) \approx 3.30$?

If the network were outputting a perfectly uniform distribution, NLL would be $\log(27) \approx 3.30$ — same as Part 1's untrained baseline. We get 17.77 because the random initialization happens to assign *very high* probability to *wrong* characters and *tiny* probability to the correct ones. The logits are scattered with large magnitudes, so after softmax the distribution is sharp and confidently wrong. The first few gradient steps will fix this dramatically — loss will fall from 17 to around 3 almost instantly. (Part 3 addresses this initialization problem properly.)

15. Making it respectable — parameter lists and generators

Karpathy now consolidates the prototype into clean code. Two new conventions:

Cell — Reproducible initialization with a generator

```
g = torch.Generator().manual_seed(2147483647)
C = torch.randn((27, 2), generator=g)
W1 = torch.randn((6, 100), generator=g)
b1 = torch.randn(100, generator=g)
W2 = torch.randn((100, 27), generator=g)
b2 = torch.randn(27, generator=g)
parameters = [C, W1, b1, W2, b2]
```

- **Pass the generator to every random op.** Without this, every cell re-run gives different numbers and you cannot reproduce results. 2147483647 is the same seed Karpathy used in Part 1 (it is $2^{31} - 1$).
- **Cluster parameters into a list.** Now we can iterate over parameters when zeroing gradients or applying updates — one loop handles all five tensors. This is the precursor to PyTorch's `nn.Module.parameters()` idiom.

Cell — Count total parameters

```
sum(p.nelement() for p in parameters)
# 3481 — about 3.5k parameters
```

`.nelement()` returns the number of elements in a tensor. Summing across all parameters gives the total parameter count of the model. 3.5k for the first version — tiny. By the end of the notebook we will be around 12k.

16. `F.cross_entropy` — efficiency and numerical stability

Cell — Replace four lines with one

```
# Before: manual softmax + NLL
# counts = logits.exp()
# prob = counts / counts.sum(1, keepdims=True)
# loss = -prob[torch.arange(32), Y].log().mean()
```

```
# After:
F.cross_entropy(logits, Y)
```

`F.cross_entropy` takes raw logits (not probabilities!) and integer targets, and returns the average NLL. One call replaces the whole softmax-then-log pipeline. There are three reasons to always prefer it over rolling your own:

Reason 1 — Fused kernel (speed and memory)

Our manual version creates three intermediate tensors: `counts`, `prob`, and the gathered probabilities. Each one allocates memory. `F.cross_entropy` fuses these into a single CUDA kernel that streams through the data once. No intermediates, much less memory pressure, much faster.

Reason 2 — Simpler backward pass

The forward pass of softmax+log+gather is complicated. Backpropping through each step would compute gradients for the intermediates we never need. But mathematically, the gradient of cross-entropy with respect to logits simplifies to a beautiful one-liner: `softmax(logits) - one_hot(Y)`. PyTorch implements this analytical form directly, skipping all the intermediate gradient computations. Just like the 1 - $\tanh^2$ trick from micrograd.

Reason 3 — Numerical stability

This is the killer reason. Imagine during training a logit grows to +100. Then $\exp(100) \approx 2.7 \times 10^{43}$, which overflows float32 and becomes `inf`. Dividing `inf` by `inf` gives `nan`, and your training collapses irrecoverably.

`F.cross_entropy` uses the “log-sum-exp” trick: before exponentiating, it subtracts the max logit from every logit. Mathematically this does not change the softmax (the same constant cancels in the numerator and denominator), but it caps the largest exponent at $\exp(0) = 1$, so nothing can overflow. Negative

logits can still go to $\exp(-\infty) = 0$, but that is harmless — we just get a tiny probability, not a NaN.

Rule of thumb

Whenever PyTorch ships a fused function for something you would implement by hand (cross-entropy, log-softmax, layer norm, attention), use it. The fused version will be faster, more memory-efficient, and numerically safer. The only exception is when you are learning — like in this notebook — and want to see the underlying math.

17. The first training loop — overfitting on purpose

Before scaling to the full dataset, Karpathy trains on just the 32 examples we have (from the first 5 words) to prove the optimization machinery works.

Cell — Enable gradients on all parameters

```
for p in parameters:
    p.requires_grad = True
```

Cell — The training loop

```
for i in range(1000):
    # forward
    emb = C[X]                                # (32, 3, 2)
    h = torch.tanh(emb.view(-1, 6) @ W1 + b1)  # (32, 100)
    logits = h @ W2 + b2                      # (32, 27)
    loss = F.cross_entropy(logits, Y)

    # backward
    for p in parameters:
        p.grad = None
    loss.backward()

    # update
    for p in parameters:
        p.data += -0.1 * p.grad

print(loss.item())
# something tiny, like 0.25
```

The pattern is the same as Part 1, generalized to multiple parameter tensors:

- Forward pass — embed, hidden layer, output, loss.
- Backward pass — clear gradients (`p.grad = None`), then call `loss.backward()`. PyTorch walks the graph from loss back to every parameter with `requires_grad=True`.
- Update — nudge each parameter in the direction opposite its gradient. Learning rate 0.1, picked by gut.

Why does Karpathy deliberately overfit 32 examples?

It is a sanity check that the network has enough capacity to fit something. 3481 parameters trained on 32 examples can memorize them — loss should drop to near zero. If it cannot even overfit a tiny dataset, there is a bug in your forward pass. This is a debugging move every ML practitioner does.

Why loss does not go to exactly zero

Loss settles around 0.25, not 0.0. The reason: in the first 5 words, the context [. , . , .] appears five times — once for each word's first character. But the “correct” next character differs: *e* for emma, *o* for olivia, *a* for ava, *i* for isabella, *s* for sophia. The same input has five different targets, so no setting of the weights can give probability 1 to all five. The lowest the loss can go on those examples is the entropy of that target distribution. The remaining examples drag the average loss down to 0.25.

18. Scaling up: full dataset of 228k bigrams

Rebuild the dataset using all of words, not just the first 5. The dataset grows from 32 to 228,146 examples. Same model, same training loop — we just run it on the full data.

The first issue: speed. Every iteration now embeds, matmuls, and backprops on 228k examples. That is slow.

19. Minibatches with torch.randint

The fix is one of the load-bearing tricks of deep learning: do not compute the gradient on the entire dataset. Sample a small random subset (a **minibatch**), compute the gradient on just that, take a step, repeat.

Cell — Sample a minibatch

```
ix = torch.randint(0, X.shape[0], (32,))
# 32 random integer indices into the dataset, with replacement
```

- `torch.randint(0, N, (32,))` returns 32 random integers in the range [0, N). These are the indices of the examples we will use this step.
- `X[ix]` grabs those 32 rows from X; `Y[ix]` grabs the corresponding targets.
- **The forward pass is identical** — just on the 32-row subset instead of the full 228k.

In your notebook this becomes the standard form of the inner loop:

```
ix = torch.randint(0, X.shape[0], (32,))
emb = C[X[ix]] # (32, 3, 2)
h = torch.tanh(emb.view(-1, 6) @ W1 + b1)
logits = h @ W2 + b2
loss = F.cross_entropy(logits, Y[ix])

for p in parameters: p.grad = None
```

```
loss.backward()
for p in parameters: p.data += -lr * p.grad
```

Approximate gradient on 32 examples vs exact gradient on 228k

“The gradient direction is good enough, even when it is estimated on only 32 examples, that it is useful. It is much better to have an approximate gradient and make many steps than it is to evaluate the exact gradient and make few steps.” — Karpathy. The minibatch direction is noisy, but cheap. Many noisy steps beat few precise ones. Your notebook puts it concisely: *approx direction and take steps >>> exact direction and take the steps*.

20. The learning rate finder

We have been guessing learning rates. Karpathy now shows a clever empirical trick to *find* a good one. The idea: sweep through many candidate learning rates exponentially, plot loss as a function of learning rate, and read off the value where loss is still falling fast but not yet exploding.

Cell — Build a logarithmically-spaced range of learning rates

```
lre = torch.linspace(-3, 0, 1000)
lrs = 10**lre
```

Reading this carefully:

- `torch.linspace(-3, 0, 1000)` gives 1000 numbers evenly spaced between -3 and 0 . Call these the “learning rate *exponents*.”
- `10**lre` raises 10 to each one, giving learning rates that are exponentially spaced between $10^{-3} = 0.001$ and $10^0 = 1$.
- **Why exponentially spaced?** Because intuition about learning rates is multiplicative, not additive. The interval $[0.001, 0.01]$ is the same kind of jump as $[0.01, 0.1]$ — both $10\times$. Linear spacing would waste most of the candidates near 1.0 .

Cell — Sweep through the learning rates while training

```
lri, lossi = [], []

for i in range(1000):
    ix = torch.randint(0, X.shape[0], (32,))

    # forward
    emb = C[X[ix]]
    h = torch.tanh(emb.view(-1, 6) @ W1 + b1)
    logits = h @ W2 + b2
    loss = F.cross_entropy(logits, Y[ix])

    # backward
    for p in parameters: p.grad = None
    loss.backward()

    # update — using a DIFFERENT learning rate each step
    lr = lrs[i]
```

```

for p in parameters:
    p.data += -lr * p.grad

# log the LR and the loss it produced
lri.append(lrs[i])          # exponent, for nicer plotting
lossi.append(loss.item())

```

Step i uses learning rate $lrs[i]$. Step 0 uses 0.001 (basically does nothing), step 999 uses 1.0 (probably explodes). We track each LR exponent and the resulting loss.

21. Reading the LR-vs-loss plot — choosing 0.1

Cell — Plot

```
plt.plot(lri, lossi)
```

You see a curve with three regions:

- **Left side (very small LR, exponent near -3):** loss barely moves. The model is being nudged so gently it cannot escape its initial bad guess.
- **Middle (exponents around -1):** loss falls steeply. This is the sweet spot.
- **Right side (exponent near 0, LR near 1):** loss is chaotic, jumping around, or rising. The updates are so large they overshoot the loss landscape.

The right learning rate is in the middle of the “loss falling steeply” region. From the plot, the exponent is around -1, which means $LR \approx 10^{-1} = 0.1$. Your notebook records this finding: *“as we can see -1 i.e. -0.1 was a very good lr, which we are gonna keep.”*

The LR-finder technique in production

This trick is sometimes called the “Leslie Smith LR finder” (introduced in a 2015 paper and popularized by fast.ai). For a tiny model like ours, 1000 sweep steps takes seconds. For a large transformer it would take longer, but the technique scales fine if you sweep over a few hundred steps with a small batch. Beats guessing every time.

22. Learning rate decay

Train with $LR=0.1$ for many thousand steps and the loss falls quickly, then starts to oscillate around some plateau. The fix is the second big LR trick: **learning rate decay**. After most of the descent is done, drop the learning rate by 10× (to 0.01) and run a few more thousand steps. The smaller step size lets the optimizer settle into a better minimum it was previously overshooting.

Cell — Decay at the end

```

# during the final phase of training:
lr = 0.01    # was 0.1
for p in parameters:
    p.data += -lr * p.grad

```


Your notebook captures the intuition: “at the last of the training stages make 0.1 to 0.01, just to slow down at the end as we near the min loss.” Production training uses more sophisticated schedules (cosine, linear warmup + decay, ...), but the principle is the same: large steps to find the basin, small steps to settle in it.

23. The train / dev / test split — what each one is for

At this point the notebook hits its first real plateau and Karpathy delivers a key methodological lesson. Your notebook frames the problem perfectly:

The problem you wrote down

“Since dataset is small, it is memorising it, which leads to overfitting. We get to know that when we try to sample datasets from it, we only get the same examples which were provided to you. To deal with this we split our dataset into training set, dev/validation set (used over the hyperparameters of the model) and test set.”

The three splits and their disjoint jobs:

Split	Size	Used for
Train	80%	Updating parameters with gradient descent.
Dev (validation)	10%	Comparing different models/hyperparameters (LR, hidden size, embedding dim, block size)
Test	10%	Final, single-use evaluation. If you tune on this, you have leaked the test set into your decision

Cell — Build the splits

```
def build_dataset(words):
    block_size = 3
    X, Y = [], []
    for w in words:
        context = [0] * block_size
        for ch in w + '.':
            ix = stoi[ch]
            X.append(context)
            Y.append(ix)
            context = context[1:] + [ix]
    X = torch.tensor(X)
    Y = torch.tensor(Y)
    print(X.shape, Y.shape)
    return X, Y

import random
random.seed(42)
random.shuffle(words)
n1 = int(0.8 * len(words))
n2 = int(0.9 * len(words))

Xtr, Ytr = build_dataset(words[:n1])      # 80% — train
Xdev, Ydev = build_dataset(words[n1:n2])  # 10% — dev
Xte, Yte = build_dataset(words[n2:])      # 10% — test
```

- `random.seed(42)` — seed Python's PRNG (separate from PyTorch's) so the shuffle is reproducible. 42 is a famous arbitrary number from Hitchhiker's Guide; people use it everywhere.
- `random.shuffle(words)` — modifies words in place. Critical: the file was sorted by name frequency. Without shuffling, your train set would be common names and your test set would be rare names — a distribution shift that would make dev/test loss artificially high.
- **The function** `build_dataset` — same windowing logic as before, factored into a function so we can call it on each split.

24. Underfitting: when train loss equals dev loss

After training the model on the train split, we evaluate on both train and dev:

Cell — Evaluate on train

```
emb = C[Xtr]
h = torch.tanh(emb.view(-1, 6) @ W1 + b1)
logits = h @ W2 + b2
loss = F.cross_entropy(logits, Ytr)
loss # something like 2.30
```

Cell — Evaluate on dev

```
emb = C[Xdev]
h = torch.tanh(emb.view(-1, 6) @ W1 + b1)
logits = h @ W2 + b2
loss = F.cross_entropy(logits, Ydev)
loss # also about 2.30
```

Two scenarios to read off this comparison:

Scenario	What it looks like	What it means	What to do
Underfit	Train loss $\approx$ Dev loss, both still high	The model is too small — it can't fit the training data well, so it can't generalize	Make the model bigger (more hidden units, more layers)
Overfit	Train loss low, Dev loss much higher	The model has memorized the training data but does not generalize	Regularize, get more data, or shrink the model.
Just right	Both losses fall together, dev slightly higher	The model is well-matched to the data	Stop training, ship.

Your notebook reads off the diagnosis correctly: “Training loss and the dev loss come to be similar, case of underfitting, which indicates our network is very tiny, very small, so let's fix that.” The fix is in the next chapter.

25. First scale-up: 100 $\rightarrow$ 300 hidden neurons

First lever to pull: more hidden units. Triple them.

Cell — Bigger hidden layer

```
C = torch.randn((27, 2), generator=g)
W1 = torch.randn((6, 300), generator=g) # was (6, 100)
b1 = torch.randn(300, generator=g) # 300 biases
```

```
w2 = torch.randn((300, 27), generator=g) # 300 inputs into output
b2 = torch.randn(27, generator=g)
parameters = [C, w1, b1, w2, b2]
```

Parameter count jumps to about 10k. Train for 30,000 minibatch steps with LR 0.05 (a bit smaller, since the bigger model is more sensitive). Loss falls, but not by much. We also plot the loss curve:

Cell — Plot the loss over training

```
plt.plot(stepi, lossi)
```

The “hairy band” in the loss curve

Your notebook notes: “The thickness in the bottom signifies the work done on minibatches.” Spot on. The loss curve looks like a thick fuzzy line, not a smooth descent. Each step's loss is computed on a different random minibatch, so the loss values bounce up and down by chance. The *trend* is downward, but each individual point is noisy. This is normal minibatch SGD behaviour. If you want a cleaner curve, plot a running average.

Dev loss barely moves. Karpathy diagnoses the new bottleneck: it is not the hidden layer anymore, it is the 2-D embedding. Twenty-seven characters squeezed into a 2-D space cannot be discriminated finely enough for the rest of the network to use.

26. Visualizing the 2-D embedding space

Before increasing the embedding dimension, the notebook pauses to *look* at what the network learned in 2-D. This is one of the most rewarding cells in the whole video.

Cell — Scatter plot of all 27 character embeddings

```
plt.figure(figsize=(8, 8))
plt.scatter(C[:, 0].data, C[:, 1].data, s=200)
for i in range(C.shape[0]):
    plt.text(C[i, 0].item(), C[i, 1].item(), itos[i],
            ha='center', va='center', color='white')
plt.grid('minor')
```

What you will see in the plot:

- **The vowels cluster together.** a, e, i, o, u end up in roughly one region of the 2-D space. The network discovered, without being told, that vowels play similar grammatical roles in name spelling.
- **The special token .** usually sits off on its own, away from the letters — it has its own special role (start/end), and the network learned it deserves its own region.
- **Other consonant groups** form looser clusters too, often by phonetic similarity.

Why this is the “wow” moment of the lecture

Nobody told the network what vowels are. Nobody encoded any linguistic structure. We just minimized next-character prediction loss with gradient descent, and out fell a representation that puts vowels next to each other. **This is what learned embeddings give you.** Scale this up to GPT and you get embeddings where *king – man + woman ≈ queen*. Same machinery, more data.

But the plot also shows the limitation: 27 distinct things crammed into 2-D inevitably overlap. We can see the network *wants* finer distinctions but does not have the room. Time to give it more room.

27. Second scale-up: 2-D → 10-D embeddings

We give each character a 10-dimensional embedding instead of 2. The input to the hidden layer is now $3 \times 10 = 30$ numbers per example, so W_1 needs to be $(30, \cdot)$. We trim the hidden layer back to 200 since the bigger embedding picked up some of the slack.

Cell — The bigger model

```
C = torch.randn((27, 10), generator=g) # 2 → 10
W1 = torch.randn((30, 200), generator=g) # input 3*10=30, 200 hidden
b1 = torch.randn(200, generator=g)
W2 = torch.randn((200, 27), generator=g)
b2 = torch.randn(27, generator=g)
parameters = [C, W1, b1, W2, b2]
```

Forward pass changes only in the `.view` call:

```
emb = C[Xtr[ix]] # (32, 3, 10)
h = torch.tanh(emb.view(-1, 30) @ W1 + b1) # 30, not 6
logits = h @ W2 + b2
loss = F.cross_entropy(logits, Ytr[ix])
```

We train for 50,000 minibatch steps at LR 0.01. The loss curve is now plotted on a $\log_{10}$ scale so the late-training plateau is visible:

```
lossi.append(loss.log10().item())
```

Your notebook explains it well: “Let’s plot the log loss instead of normal loss, as log squashes everything in the graph.” Without the log, the early steep drop dominates the plot and you cannot see what is happening at the end.

28. Beating the bigram baseline — and the first sign of overfitting

Final evaluation on both splits:

Cell — Train and dev loss

```
# Train
emb = C[Xtr]
h = torch.tanh(emb.view(-1, 30) @ W1 + b1)
```

```
logits = h @ W2 + b2
loss = F.cross_entropy(logits, Ytr)
loss    # about 2.18

# Dev
emb = C[Xdev]
h = torch.tanh(emb.view(-1, 30) @ W1 + b1)
logits = h @ W2 + b2
loss = F.cross_entropy(logits, Ydev)
loss    # about 2.20
```

Model	Loss
Bigram (counting)	2.45
Bigram (neural)	2.47
MLP, 2-D emb, 100 hidden	~2.30 (underfit)
MLP, 2-D emb, 300 hidden	~2.27 (still 2-D bottleneck)
MLP, 10-D emb, 200 hidden	2.18 train, 2.20 dev ✓

Your notebook records the win and the warning shot: “we beat the 2.35 mark by having 2.20 and also the train loss and val loss are starting to depart, which means overfitting just started.”

Reading the train/dev gap

Train 2.18, dev 2.20. The model is now *slightly* better on training data than dev data — the gap is small (0.02) but it is the first time it has appeared. This is the boundary where you stop scaling up blindly and start thinking about regularization and more sophisticated tricks. That is what Part 3 of the series is about: better initialization, batch normalization, and diagnostic plots for activations and gradients.

29. Summary and key takeaways

What we built

A character-level MLP language model following Bengio et al. 2003. Each character is mapped to a learned 10-D embedding, three consecutive embeddings are concatenated into a 30-D context vector, passed through a 200-unit hidden layer with tanh non-linearity, and finally projected to 27 logits. Trained with minibatch SGD on 228k bigrams. Loss: 2.20 on dev, vs 2.45 for the bigram — a real improvement.

The seven concepts that matter most

- 1. Embeddings.** Each token gets a learned dense vector. Indexing a matrix is mathematically the same as multiplying a one-hot vector by it, but vastly more efficient. Same primitive as Word2Vec, GloVe, BERT, GPT — all the way up.

- **2. `.view()` is free.** Changing the shape of a tensor (without changing the data) is metadata-only in PyTorch. Reach for `.view` before `cat` or `unbind`. The `-1` trick handles unknown batch sizes elegantly.
- **3. Always use `F.cross_entropy`.** It is a fused kernel that is faster, more memory-efficient, and numerically stable thanks to the log-sum-exp trick. Never roll your own softmax+log in production.
- **4. Minibatches beat full-batch.** Approximate gradients are cheap. Cheap noisy gradients you can take 1000 of beat one exact gradient you took an hour to compute. Standard size: 32–256. The variance in the gradient even acts as a mild regularizer.
- **5. The LR finder.** Do not guess the learning rate. Sweep exponentially from $1e-3$ to 1, plot loss, pick the value in the steep-descent region. Two minutes of work that saves hours of bad runs.
- **6. LR decay.** Once loss plateaus, drop the learning rate by $10\times$ and keep training. The smaller step lets the optimizer settle into a minimum it was overshooting.
- **7. Train / dev / test.** Without these three disjoint splits you literally cannot tell if your model is good or just memorising. Train updates parameters; dev compares models; test evaluates exactly once.

Diagnostic flowchart

Memorize this. It comes up in every project after this one.

- **Train loss high, dev loss high, they are equal → underfit.** Make the model bigger.
- **Train loss low, dev loss much higher → overfit.** Regularize, get more data, or shrink the model.
- **Train loss low, dev loss only slightly higher → healthy.** Stop.
- **Loss is NaN → numerical instability.** Check for missing `F.cross_entropy`, exploding logits, or a learning rate way too high.
- **Loss is not moving → learning rate too small or no `requires_grad`.** Sanity check both before anything else.

Where Part 3 picks up

Part 3 (*Building makemore Part 3: Activations & Gradients, BatchNorm*) takes this exact model and asks: why is the initial loss 17 instead of $\log(27) \approx 3.3$? Why does the tanh layer saturate so badly at the start? Once you see the activation and gradient distributions, the answer becomes obvious — and the fix (Kaiming init, BatchNorm) is a couple of one-liners that drop the final loss further with no architecture change. Same model, smarter setup.

The single thing to remember if you remember nothing else

Embeddings + hidden layer + softmax + cross-entropy + minibatch SGD *is* the deep-learning loop. Every model from here to a Transformer is a more elaborate forward pass wrapped around exactly this scaffold. If you understand this notebook, the rest of the series is “same loop, fancier forward pass.”

30. Glossary

Embedding. A learned dense vector representation of a discrete token (here, a character). The embedding matrix C is one of the trainable parameters of the model. Indexing into it ($C[ix]$) is mathematically equivalent to multiplying a one-hot vector by it, but much faster.

Block size / context length. How many previous tokens the model sees when predicting the next one. `block_size = 3` in this notebook.

Hidden layer. An intermediate layer between input and output. In this MLP, it is a single linear layer of size 100 / 200 / 300, followed by a `tanh` non-linearity.

Non-linearity (activation function). A function applied element-wise between linear layers, like `tanh` or `ReLU`. Without one, stacking linear layers is equivalent to a single linear layer.

Logits. The raw, pre-softmax output of a classifier. Real-valued, can be positive or negative. Interpreted as “log-counts.”

Softmax. $\exp(x) / \sum(\exp(x))$. Turns logits into a probability distribution.

Cross-entropy loss. Same thing as average NLL when targets are class indices.

`F.cross_entropy(logits, Y)` in PyTorch fuses softmax, log, and NLL into one numerically stable call.

Log-sum-exp trick. Subtracting the max value from every logit before exponentiating, so the largest $\exp$ is $\exp(0) = 1$. Does not change the result mathematically (the constant cancels) but prevents `inf` from overflow. Used by `F.cross_entropy`.

Minibatch. A small random subset of the training data used for one gradient step. Standard sizes are 32–256. The full dataset is split into many minibatches; one epoch is one full pass through all of them.

Epoch. One complete pass through the entire training set. In this notebook we do not strictly do epochs — we just sample minibatches with replacement for a fixed number of steps.

`torch.randint`. Generates random integers in a given range. Used to sample minibatch indices.

Learning rate (LR). The size of each gradient step. The most-tuned hyperparameter in deep learning.

LR finder. An empirical technique: sweep through exponentially-spaced learning rates and plot loss vs LR to find the steepest-descent region. Pick the LR in that valley.

LR decay / schedule. Lowering the learning rate during training. A common pattern is “LR=0.1 for most of training, then 0.01 at the end” — used in this notebook.

`torch.linspace`. Generates a 1-D tensor of equally spaced points between two endpoints. `torch.linspace(-3, 0, 1000)` gives 1000 points from -3 to 0.

`.view()`. Reinterpret a tensor's shape without copying data. Requires the tensor to be contiguous in memory. Use -1 for one dimension to have PyTorch infer it.

`.reshape()`. Like `.view` but falls back to a copy if the tensor is not contiguous. Slightly safer, slightly less explicit.

`torch.cat`. Concatenate a sequence of tensors along a given dimension. All tensors must have the same shape except along that dimension.

`torch.unbind`. Remove a dimension by splitting along it into a tuple of slices. Inverse of `torch.stack`.

Train / dev / test split. Three disjoint subsets of the data: train (for gradient steps), dev/validation (for hyperparameter selection), test (for one-shot final evaluation). 80/10/10 is a common ratio.

Underfitting. Model performance is poor on both train and dev sets, with both losses similar. The model is too small to capture the data.

Overfitting. Train loss is much lower than dev loss. The model has memorized training data but does not generalize.

Parameter count. Sum of `.numel()` over all trainable tensors. For our final model: about 12k. For GPT-3: 175 billion. Same equation, different scale.

Reproducibility (seed). Setting a fixed seed in random-number generators so that re-running the same code produces identical results. Different seeds for different RNGs (PyTorch and Python's `random`) must each be set.

Bengio 2003. *A Neural Probabilistic Language Model* by Bengio, Ducharme, Vincent, Jauvin. The paper that introduced learned word embeddings and the MLP language-modelling blueprint we follow in this notebook.

End of Part 2. Part 3 keeps the same network and zooms in on the internals — activation statistics, gradient flow, dead neurons, and BatchNorm. Same model, same loop, sharper microscope.